normalized to lowercase before being stored in the database.

Finally, you need to restart your server as a new gem was installed by the authentication generator.

With this code in place, we have the ability to present both a password and a password confirmation field in a form, as well as the ability to authenticate a user, given a name and a password. Not bad for two commands and three lines of code.

But now we have an embarrassing problem: there are no administrative users in the database, so we can't log in.

Fortunately, we can quickly add a user to the database from the command line. If you invoke the `rails console` command, Rails invokes Ruby's `irb` utility, but it does so in the context of your Rails application. That means you can interact with your application's code by typing Ruby statements and looking at the values they return.

We can use this to invoke our user model directly, having it add a user into the database for us:

```
depot> bin/rails console
Loading development environment (Rails 8.0.2)
work(dev)* User.create(name: "dave",
work(dev)>   email_address: "dave@example.org", password: "secret")
work(dev)> exit
```

With this in place, we can now log in as the user `dave` with the password `secret`: